

UnifAI — Administrator Guide

Audience: platform admins, IT security, compliance owners

Baseline: repo commit **16d4c3e**

1. Roles you will work with

Role	Data scope	Typical use
admin	all-data	Full control: providers and master API keys, system settings, users, RBAC, everything
sub_admin	all-data	Workspace manager: virtual keys, budgets, prompts, logs, Browser AI , MCP tool groups — but not master provider keys or system settings
user	own-data	Read/View only, on Dashboard , Logs , Inference , PromptRepository , Observability , MCPGateway . The UI label says “Prompt Repository only”, but the backend also permits read-only log/observability/MCP views — narrow this with Allowed Sections (allowed_sections) so the user's sidebar matches your intent
<i>custom role</i>	configurable	Any subset of the 33 resources × 6 operations, created by an admin

Two authorisation layers apply to every API call:

1. **Resource × Operation** (e.g. **Logs: Update**, **VirtualKeys: Create**).
2. **Sidebar section grants** (**allowed_sections** on the user) for section-scoped sessions.

Important: all **/api/browser-ai/*** routes are gated by the **Logs** resource. So to let someone manage Browser AI, give them **Logs** view/update (or the **browser-ai** section grant) — **sub_admin** already has it.

2. First-run setup checklist

1. **Sign in** with the bootstrap admin from **.env** (**ADMIN_EMAIL** / **ADMIN_PASSWORD**).
2. **Change the admin password** immediately, and create a named admin account for daily use (Governance → Users), then keep the bootstrap account for emergencies only.
3. **Configure SMTP** (**/api/smtp-config** → Settings) so password reset, registration codes and Guard warning e-mails work.
4. **Verify secrets** are long and random: **UNIFAI_ENCRYPTION_KEY**, **UNIFAI_GUARD_SECRET**, **PASSWORD_RESET_SECRET**, **CLUSTER_REPLICATE_SECRET**.
5. **Set branding** (**UNIFAI_COMPANY_NAME**, **UNIFAI_COMPANY_LOGO**).
6. **Add providers and keys** (Models → Model Providers) and confirm the model catalog populated.
7. **Create virtual keys** for teams/applications with budgets and rate limits.

8. **Set up Browser AI:** import Target Websites, review/import Guard Rules, set Controls and the company **uninstall key**, then distribute the Guard package from Setup.
9. **Configure alert channels** (webhooks) for the events you care about.
10. **Confirm Audit Logs** are recording your mutating actions (they are automatic).
11. **Schedule backups** (see SERVER.md §9.4) and test a restore.
12. **Verify health** — **GET /health** returns 200 and Guard agents appear under Browser AI → Guard Agents.

3. User and access administration

3.1 Approving and managing users

Governance → **Users** (API: **/api/session/users***):

Task	How
Approve a pending sign-up	Users → pending user → Approve (POST /api/session/users/{id}/approve)
Reject a request	Reject (POST /api/session/users/{id}/reject)
Create a user directly	Create user (POST /api/session/users)
Change role	Set role to user / sub_admin / admin , or assign a custom RBAC role (PUT /api/users/{id}/role)
Edit details	PUT /api/session/users/{id} — e-mail, budget, rate limit, allowed prompt repos, allowed sections
Disable / remove	DELETE /api/session/users/{id} (or disable via SCIM status)

User states: **pending**, **approved**, **rejected**, **email_unverified**, **disabled**.

Note: the **Users** page requires the **admin/sub_admin** role in addition to the **Users** RBAC resource — custom roles cannot open it.

3.2 Roles and permissions

Governance → **Roles & Permissions** (**/api/roles***, **/api/permissions**, **/api/resources**, **/api/operations**):

- The three **system roles** (**admin**, **sub_admin**, **user**) are seeded automatically and kept in sync on every boot (new permissions are merged in). Customisations you make are preserved, except for **admin**, which always keeps the full catalog.
- Create a **custom role** with a DAC (**all-data** / **own-data**) and a permission set.
- **Resource catalog (33):** `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning, Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins, MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance, RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys, Inference, Metrics, FeatureFlags, CircuitBreaker`
- **Operations (6):** **Read**, **View**, **Create**, **Update**, **Delete**, **Download** (**View** ≈ **Read**).

- **Scope grants** (**/api/rbac/scope-grants**) limit which records a role can reach.

3.3 Section-scoped access

For a **user**, "Allowed Sections" selects which sidebar areas they can open

(**observability**, **browser-ai**, **models**, **mcp-gateway**, **plugins**, **governance**, **guardrails**, **cluster-config**, **adaptive-routing**, **prompt-repository**, **skills-repository**, **settings**).

Parent grants cover children; **observability/llm-logs** is a child grant; the legacy

observability/browser-ai counts as the top-level **browser-ai**.

3.4 Access profiles

Governance → **Access Profiles** (**/api/access-profiles***): reusable, versioned policy templates

(name, description, tags, spec, calendar-aligned). Activate/deactivate/clone them and apply them to

users. Useful for standard personas (contractor, finance, engineering).

3.5 Teams, business units and customers

- **Teams** — group users; assign budgets and rate limits per team.
- **Business Units** — group teams under an org unit with its own budget and rate limits.
- **Customers** — external/tenant grouping for billing and reporting.
- **SCIM / User Provisioning** (**/api/scim/***) — connect your IdP (Azure AD/Okta) so joiner/mover/leaver is automatic; SCIM **active=false** maps to user status **disabled**.

4. Virtual keys, budgets and limits

Governance → **Virtual Keys** (**/api/governance/virtual-keys***):

Task	Endpoint
List keys	GET /api/governance/virtual-keys
Create a key	POST /api/governance/virtual-keys
Rotate keys	POST /api/governance/virtual-keys/rotate
Edit a key	PUT /api/governance/virtual-keys/{id}
Delete	DELETE /api/governance/virtual-keys/{id}
Assign users	/api/governance/virtual-keys/{id}/users
Self-service quota check	GET /api/governance/virtual-keys/quota (key in header)

Practices:

- One key per application or team — never a shared personal key.
- Set a **budget** (cost in USD) and a **rate limit** (requests per minute). Limits are enforced by the governance plugin; Budgets & Limits (Models → **model-limits**, Providers → **model-limits**) shows the live values.
- Rotate on staff change or suspected leak; bulk rotation is supported.

- Keys are encrypted at rest when **UNIFAI_ENCRYPTION_KEY** is set.

5. Models, providers and routing

Area	Page / endpoint	What you do
Providers + master keys	Models → Model Providers (/api/providers* , /api/keys*)	Add provider keys (multiple keys per provider are load-balanced), set network config (timeouts, retries, proxy)
Model catalog	Models → Model Catalog (/api/models*)	See discovered/live models, enable/disable, inspect pricing
Model settings	Models → Model Settings (/api/model-configs*)	Per-model parameter defaults and overrides
Pricing	Models → Pricing Overrides / Model Settings	Override costs so budgets and reports are accurate
Complexity Router	Models → Complexity Router (/api/governance/complexity-analyzer-config)	Classify requests by complexity and route to cheaper/stronger models
Routing Rules	Models → Routing Rules (/api/governance/routing-rules , /api/governance/routing-rules/tree)	Conditional routing by model, user, team, key
Circuit Breaker	Models → Circuit Breaker (/api/circuit-breaker/*)	Fail fast on unhealthy providers/keys; reset policies
Adaptive Routing	Adaptive Routing (/api/load-balancer*)	Load-balancing strategy and per-route weights

Guiding principle: put **least privilege** in front of provider keys. **sub_admin** can see models but cannot read or change master provider keys — only **admin** can.

6. Guardrails vs Guard Rules (do not confuse them)

	Guardrails (gateway)	Guard Rules (Browser AI)
Where	Guardrails → Rules / Rule Providers	Browser AI → Guard Rules
Protects	API/inference traffic through the UnifAI gateway	Employee prompts and uploads on public AI websites
Endpoints	/api/guardrails/rules , /api/guardrails/providers	/api/browser-ai/rules*
RBAC resource	GuardrailsConfig , GuardrailsProviders	Logs
Rule types	Provider-based input/output validation	regex or ai_bot (LLM-judged, optional reference image)

Configure both: Guardrails for the governed API path, Guard Rules for the browser path.

7. Browser AI administration (the guard control plane)

Workspace → **Browser AI**, tabs: Overview, Prompt Logs, Search Logs, Guard Rules, Target Websites,

Guard Agents, Guard Insights, Setup.

7.1 Target Websites

GET|POST|PUT|DELETE /api/browser-ai/targets; bulk import via **POST /api/browser-ai/targets/import**.

Field	Meaning
domain	Host to monitor (unique) — e.g. chatgpt.com , api.anthropic.com
platform_name	Friendly platform label (ChatGPT, Claude, Gemini, ...)
monitored	Whether the domain is inspected
block_entire_website	Block the whole site instead of analysing prompts
host_role	ui , chat , file or blank (auto) — drives the proxy intercept path
status	MONITORED , PAUSED , BLOCKED

Import-ready samples in the repo: **target_websites_sample.csv / .xlsx** with columns

Domain, Platform Name, Host Role, Monitored, Block Entire Website. Multiple domains per platform are normal — include UI, chat and file hosts.

7.2 Guard Rules

GET|POST|PUT|DELETE /api/browser-ai/rules, **POST /rules/import**, **POST /rules/test-bot**, **POST /rules/generate-regex**.

Field	Values
Rule Name	any text (import key; used with "Overwrite existing")
Rule Type	regex (default) or ai_bot
Pattern	valid regex
Severity	CRITICAL , HIGH , MEDIUM
Action	BLOCK , REDACT , WARN
Warning Message	text shown to the employee when the rule hits
Active	TRUE / FALSE
Description	internal note

Excel/CSV import — accepted files **.xlsx**, **.xls**, **.csv**; row 1 must be the header row with the columns above. **ai_bot** rules **cannot be imported**; create those in the UI. Export uses the same column names, so export → edit → import is a supported round-trip. Samples:

guard_rules_import_example.csv, **guard_rules_sample.csv**,

guard_rules_enterprise_1000_all_sectors.csv (card PANs, Aadhaar, salary/HR terms, API keys/secrets, private keys, internal codenames). Review them against your own policy before enabling.

AI Guard Bot rules need an LLM: the UI defaults to the **ollama** provider with model **llama3.2**

(**/api/browser-ai/ollama-models** lists availability). Vision/reference-image rules accept a reference image up to 512 KB. Always use **test-bot** before enabling.

Rules reach agents within seconds (server cache ~2 s; PAC sync default 3 s).

7.3 Controls

GET|PUT /api/browser-ai/controls — the single-row interaction policy: upload/download warnings and the interaction toggles that apply to every monitored agent. This is where you set the wording employees see when something is blocked.

7.4 Fleet configuration

GET|PUT /api/browser-ai/fleet-config — company-wide Guard defaults stored in **browser_guard_fleet_config**. Agents pull this on heartbeat, so changes here roll out without rebuilding installers.

7.5 Guard Agents (fleet inventory)

GET /api/browser-ai/agents, filterable by status, source (**endpoint** = Laptop Guard, **network** = server proxy) and search. Each record shows hostname, agent type, last seen, health detail, contact e-mail, uninstall-key state and proxy bundle SHA.

Task	Endpoint
Delete an agent record	DELETE /api/browser-ai/agents/{id} or POST /api/browser-ai/agents/bulk-delete
Set a device contact e-mail	PUT /api/browser-ai/agents/{id}/contact-email
Company uninstall key	GET
Per-agent uninstall key / rotate	GET /api/browser-ai/agents/{id}/uninstall-key , POST .../uninstall-key/rotate
Remote uninstall	POST /api/browser-ai/agents/{id}/remote-uninstall
Agent-initiated uninstall flow	agents/uninstall-verify , agents/uninstall , agents/uninstall-ack , agents/uninstall-status

Statuses: **active**, **uninstall_pending**, **uninstalled**.

Rule of thumb: investigate any **endpoint** agent not seen for more than 7 days — that employee's protection is not reporting.

7.6 Setup and packages

Task	Endpoint
Package info / versions	GET /api/browser-ai/setup/info
Download setup ZIP	GET /api/browser-ai/setup/download.zip (-windows.zip, -mac.zip)
Rebuild & publish Guard code	POST /api/browser-ai/setup/rebuild
Rebuild history	GET /api/browser-ai/setup/rebuild-history
Hot-update bundle info / download	GET /api/browser-ai/setup/proxy-bundle.json , .../proxy-bundle.zip

After changing Guard proxy/agent Python code, run **Rebuild & Publish**; installed agents pick up the bundle automatically (SHA-tracked, with rollback of a marked-bad bundle).

For a full installer release, rebuild packages on the appropriate OS (**make build-guard-windows / make build-guard-mac**), copy them into **apps/browser-guard/release/**, and redeploy.

7.7 Prompt Logs and Search Logs (investigation surface)

- **Prompt Logs** (**GET /api/browser-ai/logs**) — filter by platform, status/action, date range and free text (prompt, platform, domain, client IP, rule, agent id/hostname). Statuses: **Allowed**, **Redacted**, **Warned**, **Blocked**, **SiteBlocked**, **Bot Answered**. A row carries the full prompt, the triggered rule, risk/predictive risk and device identity.
- **Search Logs** (**GET /api/browser-ai/search-logs**) — engine, browser, incognito flag, query, clicked URL/title, predictive risk.
- **Attachments** — **GET /api/browser-ai/attachments/{id}** (privileged).
- **Guard Insights** (**GET /api/browser-ai/insights/stats**) — per-agent action totals across the whole database.
- **Warning e-mails** — **POST /api/browser-ai/send-warning-email** sends a policy warning to an employee's contact e-mail; sends are logged in **browser_ai_warning_emails**.

Bulk cleanup: **POST /api/browser-ai/logs/bulk-delete**, **DELETE /api/browser-ai/logs** and the search-log equivalents. Deletions are destructive, admin-only and audited.

7.8 Rollout pattern (recommended)

1. Import Target Websites for the platforms you care about (start with the big 5–10).
2. Import Guard Rules in **WARN/ REDACT** mode first — do not start with blanket BLOCK.
3. Install the Guard on a pilot group; watch Prompt Logs and Guard Insights for false positives.
4. Tune regex precision, promote the critical rules to **BLOCK**.
5. Roll out to everyone; set the **uninstall key** and share it only with IT.
6. Add **AI Guard Bot** rules for nuanced cases once regex noise is under control.
7. Review weekly; publish via Rebuild & Publish when you change Guard code.

8. Observability and audit

Area	Page / endpoint	Use it for
LLM Logs	Observability → LLM Logs (/api/logs*)	Requests, tokens, cost, latency, users, virtual keys, errors
Dashboard	Observability → Dashboard (/api/logs/dashboard)	Aggregated models/cost/latency/tokens view in one call
Filters / rankings / histograms	/api/logs/filterdata , /api/logs/rankings* , /api/logs/histogram/*	Building reports, spotting outliers

Area	Page / endpoint	Use it for
MCP Logs	Observability → MCP Logs (/api/mcp-logs*)	MCP tool calls: who called what, when, result
Connectors	Observability → Connectors (/api/connectors)	BigQuery, Kafka, Datadog, New Relic, PubSub sinks
Alert Channels	Observability → Alert Channels (/api/alert-channels)	Webhook notifications (secrets encrypted at rest)
Audit Logs	Governance → Audit Logs (/api/audit-logs)	Every mutating /api/* action, captured automatically
Traces	log page trace views	End-to-end request tracing with plugin logs
Prometheus / OTel	telemetry + otel plugins	External monitoring integration

Maintenance actions: **DELETE** [/api/logs](#) (clear logs), **POST** [/api/logs/recalculate-cost](#) (recompute costs after a pricing change), **GET** [/api/logs/dropped](#) (dropped requests). All are admin-only.

9. Platform settings

Settings → sub-pages and their APIs:

Page	API	Notes
Client Settings	/api/config	Pool size, request body size, logging toggle, content logging, log retention, auth enforcement, allowed origins
Security	/api/config (security section)	Auth/security toggles
Caching	/api/cache	Cache configuration
Compatibility	/api/config (compat)	Parameter conversion between client and provider shapes
Feature Flags	/api/feature-flags	Runtime feature toggles
Logging	/api/config/logging	Which logs are recorded, retention
Observability	/api/connectors	External sinks
Performance Tuning	/api/config (performance)	Read buffer, pool sizing
Pricing Config	/api/pricing*	Default pricing data
Proxy	/api/proxy-config	Outbound proxy for provider calls
API Keys	/api/keys*	Provider keys
Plugins	/api/plugins	Enable/disable and reorder plugins
Cluster	/api/cluster	Multi-node settings (CLUSTER_REPLICATE_SECRET)
MCP Gateway	/api/mcp/settings	Gateway behaviour

GET [/api/config](#) is one of the few API routes reachable with a session in restricted setups — treat it as sensitive and keep it admin-only.

10. MCP Gateway administration

Task	Page / endpoint
Add/manage MCP clients	MCP Gateway → MCP Catalog (/api/mcp/clients , /api/mcp/client* , /api/mcp/client/{id}/reconnect)
Browse/seed the library	MCP Gateway → MCP Library (/api/mcp/library , POST /api/mcp/library/force-sync)
Group tools	MCP Gateway → Tool Groups (/api/mcp/tool-groups*)
OAuth sessions / grants	MCP Gateway → Auth Sessions (/api/mcp/sessions), OAuth Grants (/api/oauth-grants)
Auth config	MCP Gateway → MCP Auth Config (/api/mcp/auth-config*)
Tool-call logs	Observability → MCP Logs

MCP clients are dialled after plugins load; if a client's server is unavailable at boot it is retried by the health monitor. Tool groups and per-user OAuth enforce least privilege on tools.

11. Prompts, skills and prompt deployments

Area	Page / endpoint
Prompt Repository	Prompt Repository (/api/prompt-repo*)
Individual prompts	/api/prompt-repo/prompts
Lifecycle (versions, sessions)	/api/prompt-repo* lifecycle endpoints
Prompt deployments	Prompt Deployments (/api/prompt-deployments)
Skills Repository	Skills (/api/skills* , serving at /api/skills/serve/*)

Use folders and per-user [allowed_prompt_repos](#) to limit who sees which prompts.

12. Runbooks

12.1 Investigate a possible data leak

1. Browser AI → **Prompt Logs**: filter by date range and the employee/device (agent hostname or client IP).
2. Look for **Blocked**, **Redacted** or **Warned** rows; open one for the full prompt, the triggered rule and the predictive risk.
3. Check **Guard Insights** for that agent's overall behaviour and **Search Logs** for related queries.
4. If an attachment is involved, download it via [/api/browser-ai/attachments/{id}](#) and follow your evidence-handling process.
5. If warranted, send a policy warning ([/api/browser-ai/send-warning-email](#)) and escalate per your incident policy; note the Audit Log entry for anything you changed.

12.2 Block a newly popular AI tool immediately

1. Browser AI → **Target Websites** → add the domain(s) with the right **host_role** (ui/chat/file) and **monitored** set.
2. To hard-stop it, enable **Block Entire Website**; for visibility only, leave it monitored and rely on rules.
3. Allow a few seconds for the PAC/rule refresh, then test in a browser.

12.3 Onboard a department

1. Create a **Team** (and a **Business Unit** if needed) with budget/rate limits.
2. Create or assign an RBAC role (e.g. **user** + Prompt Repository) and an **Access Profile**.
3. Invite users (or let SCIM provision them); approve as needed.
4. Issue a Virtual Key for their application with a budget and rate limit.
5. Send them the Guard installers from Setup (Windows ZIP / macOS ZIP).

12.4 Rotate a leaked Virtual Key

1. Governance → Virtual Keys → **Rotate** (**POST /api/governance/virtual-keys/rotate**).
2. Distribute the new key out-of-band and confirm the old key is rejected.
3. Review LLM Logs for that key's recent usage and report per policy.

12.5 Rotate the Guard fleet secret

1. Generate a new **UNIFAI_GUARD_SECRET** (**openssl rand -base64 32**) and set it in **.env**.
2. Run **python apps/browser-guard/scripts/sync_config_from_env.py** to regenerate Guard config and docs.
3. Rebuild the Guard installers (or Rebuild & Publish) so new/updated agents carry the new secret.
4. Redeploy the backend with **UNIFAI_GUARD_REQUIRE_SECRET=1**.
5. Watch Guard Agents — agents on the old secret will fail to report until updated; plan a maintenance window and communicate it.

12.6 Turn off the Guard on a departing employee's device

1. Browser AI → **Guard Agents** → find the device → **Remote uninstall** (**POST /api/browser-ai/agents/{id}/remote-uninstall**).
2. Confirm the status moves **uninstall_pending** → **uninstalled**.
3. Disable the user account (Governance → Users), or let SCIM deprovision it.
4. Rotate any Virtual Keys they had.

12.7 Reduce LLM spend

1. Observability → **Dashboard / LLM Logs**: identify the biggest spenders (model, user, key, team).
2. Tighten budgets and rate limits on the offending keys/teams.
3. Enable/adjust the **Complexity Router** and **Routing Rules** to send simple work to cheaper models.
4. Verify pricing overrides; run **POST /api/logs/recalculate-cost** after correcting pricing.

12.8 Investigate "a rule isn't firing"

1. Confirm the site is in Target Websites and **monitored**, and the platform's **chat** host is included

(many platforms split UI/chat/file hosts).

2. Confirm the rule is **Active**, and check its regex with the **Regex live test** panel in the UI.
3. Confirm the agent is reporting (Guard Agents → last seen) and the device has no certificate warnings.
4. Remember the propagation delay: server cache ~2 s, PAC sync default 3 s.

13. Routine checklist

Daily

- [] **/health** is green
- [] Guard Agents: any **endpoint** agent not seen recently
- [] Prompt Logs: review notable **Blocked** / **SiteBlocked** events
- [] Backups completed

Weekly

- [] Guard Insights / Search Logs trend review; tune noisy rules
- [] Audit Logs spot check (role changes, key rotations, deletions)
- [] Budget/limit breaches in LLM Logs
- [] Guard rebuild/publish history if code changed

Monthly

- [] Review RBAC roles, access profiles and users (remove stale accounts)
- [] Rotate/verify the uninstall key policy with IT
- [] Pricing accuracy vs provider invoices
- [] Restore-test a backup
- [] Plugin/feature-flag review, log retention and disk check

Quarterly / on change

- [] Rotate **UNIFAI_GUARD_SECRET**, **PASSWORD_RESET_SECRET** and **UNIFAI_ENCRYPTION_KEY** per policy
- [] Re-review Guard Rules against current company policy and regulations
- [] Patch the container image, Postgres and the host

14. Do-nots

- Do not put live secrets in **configs/** or **apps/browser-guard/config/** — they are version-controlled. (**apps/browser-guard/config/unifai_guard_config.json** in this repo currently contains a live-looking **guard_secret**; rotate it and rely on **.env** plus **sync_config_from_env.py**.)
- Do not bind-mount **config.json** as a single file.
- Do not run the laptop Guard and the corporate network proxy on the same browser session.
- Do not hand out **admin** casually — **sub_admin** covers workspace management without master provider keys.
- Do not delete logs casually: it is destructive, audited and destroys evidence.
- Do not treat incognito mode as a privacy control — monitored browsers still report.
- Do not import **ai_bot** rules from Excel/CSV; that is UI-only.